

Sprawozdanie Projekt 3 OE:

Szymon Poznański

Hubert Chruścicki

Przemysław Wojnar

Wstęp

Celem projektu było zintegrowanie i przetestowanie biblioteki **PyGAD** w ramach istniejącego systemu optymalizacji ewolucyjnej, opracowanego w projektach P1 i P2. Implementacja silnika opartego o **PyGAD** została celowo ujednolicona z naszym własnym algorytmem genetycznym. Dzięki temu zachowano spójność interfejsu użytkownika (UI) oraz umożliwiono bezpośrednie porównanie parametrów i wyników. Aplikacja rozszerza wcześniej już zaprojektowaną architekturę klient-serwer jako nowoczesna aplikacja webowa Full-Stack.

PyGAD (Python Genetic Algorithm) jest biblioteką open-source przeznaczoną do implementacji algorytmów genetycznych w Pythonie. Biblioteka dostarcza gotową klasę `pygad.GA`, której zachowanie konfiguruje się za pomocą parametrów konstruktora. Biblioteka umożliwia również przekazanie własnych funkcji operatorów genetycznych jako argumentów, co zostało wykorzystane do zaimplementowania niestandardowego krzyżowania i mutacji Gaussa zgodnie ze specyfikacją projektu. PyGAD oferuje ponadto wbudowane wsparcie dla przetwarzania równoległego za pomocą odpowiedniego parametru, pozwalającego uruchomić ewaluację funkcji przystosowania na wielu wątkach jednocześnie, co znacząco skraca czas obliczeń dla populacji o dużej liczbie osobników.

Opis integracji z biblioteką PyGAD

W celu zachowania spójności, interfejs użytkownika w zakładce PyGAD obsługuje te same pola co w przypadku autorskiej implementacji (P1/P2). Mapowanie parametrów na parametry klasy `pygad.GA` przedstawia się następująco:

- **Test Function** - Wybór funkcji testowej z biblioteki `benchmark_functions`
- **Representation** - Wybór typu kodowania: rzeczywisty (*Real-valued*, mapowane na `gene_type=float`) lub binarny (*Binary*, mapowane na `gene_type=int` o zakresie `[0, 2)`).
- **Population Size** - Liczba osobników w populacji (mapowana na parametr `sol_per_pop`).
- **Number of Epochs** - Liczba generacji (mapowana na parametr `num_generations`).
- **Selection Method** - Metoda selekcji. Wspólne opcje obejmują turniejową (*tournament*) oraz koło ruletki (*rws*), zaś PyGAD dodaje opcję losową (*random*), rangową (*rank*) oraz stochastyczne próbkowanie uniwersalne (*sss*).
- **Crossover Method** - Metoda krzyżowania. PyGAD oferuje metody jednopunktowe (*single_point*), dwupunktowe (*two_points*) oraz jednorodnie (*uniform*).
- **Mutation Method** - Metoda mutacji. Wspólne opcje obejmują losową (*random*) oraz zamianę genów (*swap*).

- **Elitism Strategy** - Włączona strategia elitarna (checkbox w UI) oznacza przenoszenie najlepszego osobnika (`keep_elitism=1`), natomiast wyłączona oznacza `keep_elitism=0`.
- **Precision (Bits per Variable)** - Długość łańcucha bitów przypadająca na jedną zmienną decyzyjną (w reprezentacji binarnej).

Zgodnie z wymaganiami projektowymi, zaimplementowano również własne operatory:

1. **Własne krzyżowanie jednopunktowe** - Wybiera punkt podziału i łączy początek pierwszego rodzica z końcem drugiego:

$$\begin{aligned}\text{offspring}_1[0 \dots s] &= \text{parent}_1[0 \dots s] \\ \text{offspring}_1[s + 1 \dots L] &= \text{parent}_2[s + 1 \dots L]\end{aligned}$$

2. **Własna mutacja Gaussa** - Dla każdego potomka modyfikuje jeden losowo wybrany gen, dodając do niego losową wartość z rozkładu normalnego (0, 1).

Różnice względem poprzedniego podejścia

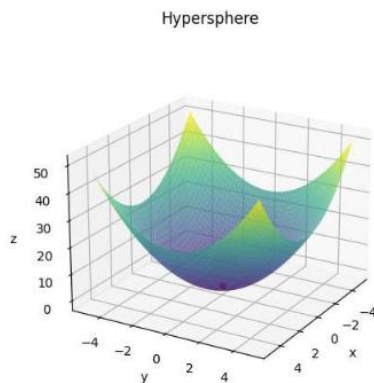
Podstawową różnicą architektoniczną względem P1/P2 jest podejście do mutacji - własny algorytm genetyczny operuje na prawdopodobieństwie mutacji per gen (`mutation_prob`), natomiast PyGAD domyślnie przyjmuje stałą liczbę genów mutowanych w każdym potomku. W ramach projektu P3 zdecydowano się jednak na użycie parametru `mutation_probability`, który przywraca podejście identyczne z P1/P2, zapewniając tym samym porównywalność eksperymentów. Kolejna różnica dotyczy wewnętrznej logiki algorytmu - PyGAD jest biblioteką maksymalizującą, dlatego dla problemów minimalizacji konieczne było zastosowanie negacji wartości funkcji celu ($-f(x)$), w odróżnieniu od własnej implementacji, która od początku obsługuje minimalizację bezpośrednio.

W każdej generacji algorytm rejestruje cztery metryki populacji oraz odchylenie standardowe (`std`). Odchylenie standardowe jest nowym kluczowym wskaźnikiem zbieżności algorytmu, jego systematyczny spadek w kolejnych generacjach świadczy o homogenizacji populacji i zbliżaniu się do optimum, natomiast nagłe wzrosty mogą sygnalizować działanie operatorów mutacji wprowadzających różnorodność genetyczną. Wszystkie cztery metryki są zbierane per generacja i prezentowane w postaci wykresów liniowych w interfejsie użytkownika, co umożliwi wizualną analizę przebiegu procesu optymalizacji i bezpośrednie porównanie z wynikami własnego algorytmu z P1/P2.

Funkcja testowa i jej optima

W projekcie zachowano funkcję sferyczną (Hypersphere) jako podstawową funkcję testową, zapewniając tym samym ciągłość i porównywalność wyników z projektami P1 i P2. Biblioteka PyGAD nie narzuca konkretnej funkcji celu. W zaimplementowanym interfejsie użytkownik ma możliwość wyboru spośród szerokiego zestawu funkcji benchmarkowych dostarczanych przez pakiet `benchmark_functions`, takich jak Rastrigin, Rosenbrock, Ackley, Schwefel, Griewank czy Styblinski-Tang. Możliwość ta może zostać wykorzystana w dalszej

zanalizie porównawczej, jednak dla zachowania spójności z poprzednimi sprawozdaniami domyślną i referencyjną funkcją testową pozostaje funkcja sferyczna. Poniżej znajdują się przypomnienie właściwości funkcji sferycznej.



- **Wzór ogólny:** wartość dostosowania obliczana jest jako suma kwadratów wszystkich zmiennych decyzyjnych danego osobnika, co można opisać wzorem:

$$f(x) = \sum_{i=1}^N x_i^2$$

- **Przyjęta dziedzina poszukiwań:** każda zmienna testowa jest standardowo inicjowana w przedziale od -5.0 do 5.0.
- **Optimum - minimum globalne:** zadanie algorytmu opiera się na problemie minimalizacji. Oczekiwane, idealne i absolutne optimum globalne - najniższa możliwa wartość funkcji wynosi dokładnie 0.0 jako wartość samej funkcji (fitness), a odnajdywane jest tylko w jednym punkcie środkowym układu współrzędnych, dla wektora decyzyjnego w którym wszystkie składowe przyjmują zerowe wartości precyzyjne: $x = [0.0, 0.0, \dots, 0.0]$

Z racji precyzji obliczeń numerycznych, dobrym i oczekiwanym rezultatem pracy algorytmu ewolucyjnego jest moment, w którym populacja we wszystkich wymiarach zbliża się możliwie najbliżej wartości idealnej wynoszącej 0.0.

Wykresy wyników

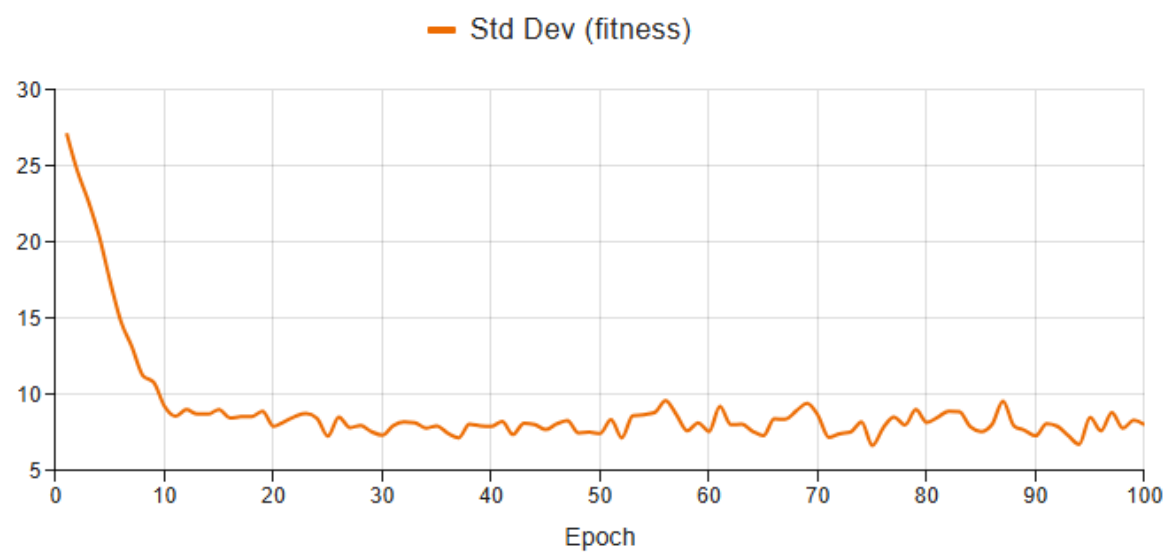
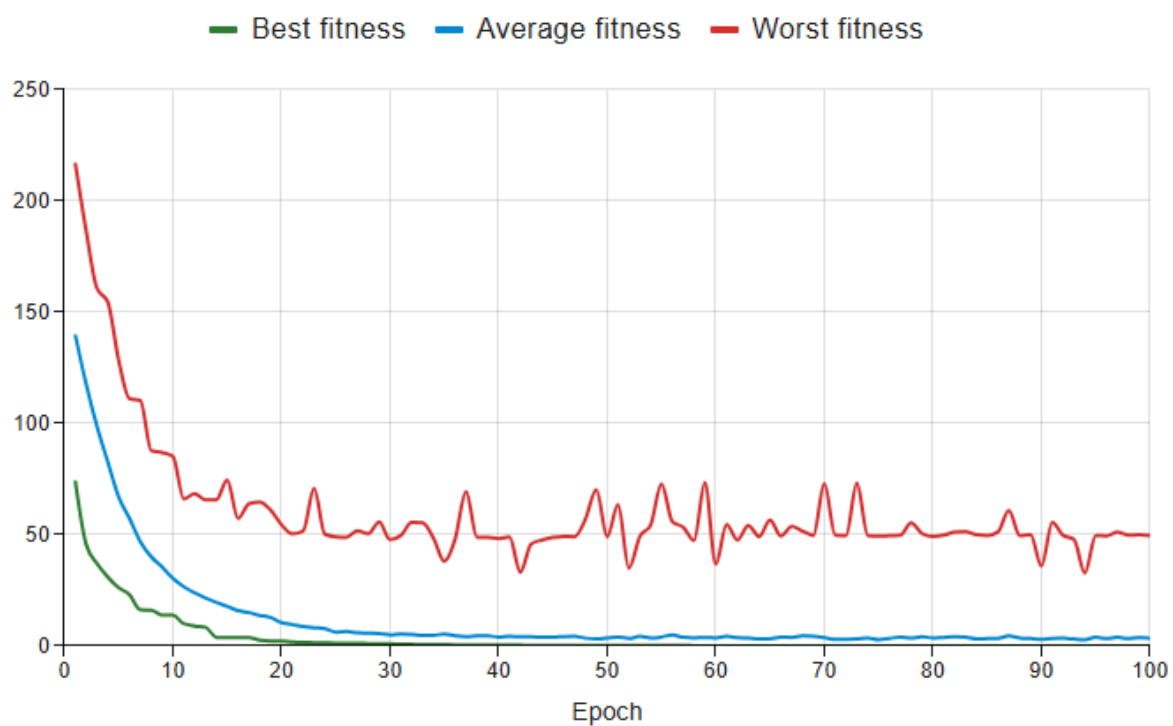
Implementacja algorytmu genetycznego z projektów P1 i P2 nie posiada wsparcia dla przetwarzania wielowątkowego. Aby zapewnić rzetelne i porównywalne warunki testowe, we wszystkich dziesięciu scenariuszach parametr Parallel Threads biblioteki PyGAD został ustawiony na wartość 0 (wyłączony). Oznacza to rezygnację z wbudowanej funkcji równoległej ewaluacji populacji oferowanej przez PyGAD, co przekłada się na dłuższy czas obliczeń po stronie P3, lecz eliminuje przewagę sprzętową jako zmienną zakłócającą wyniki.

Konfiguracja 1 - Zbalansowana

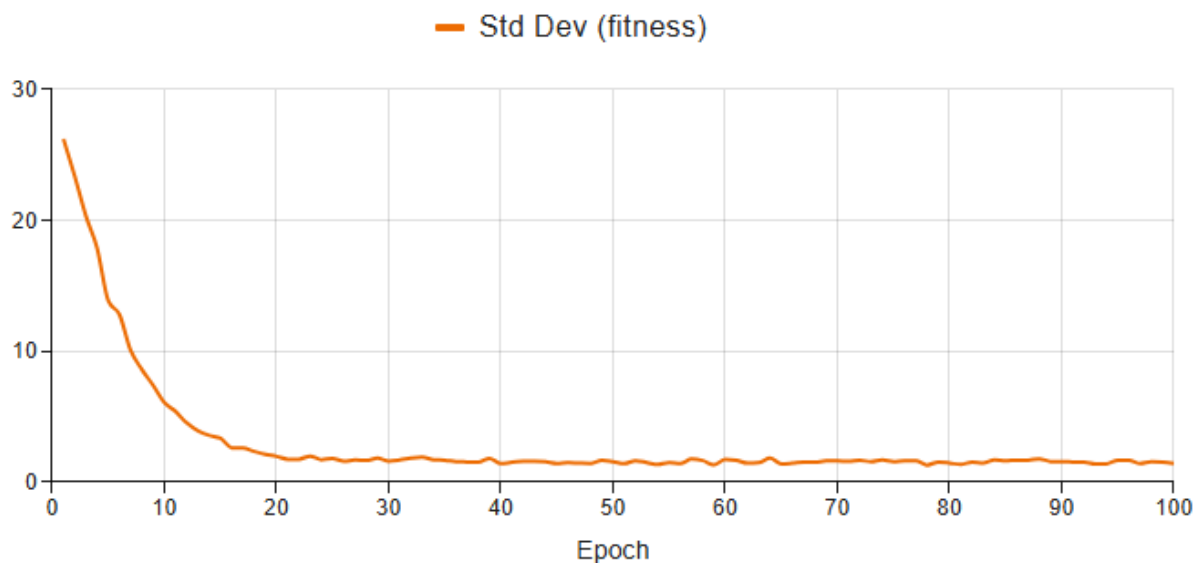
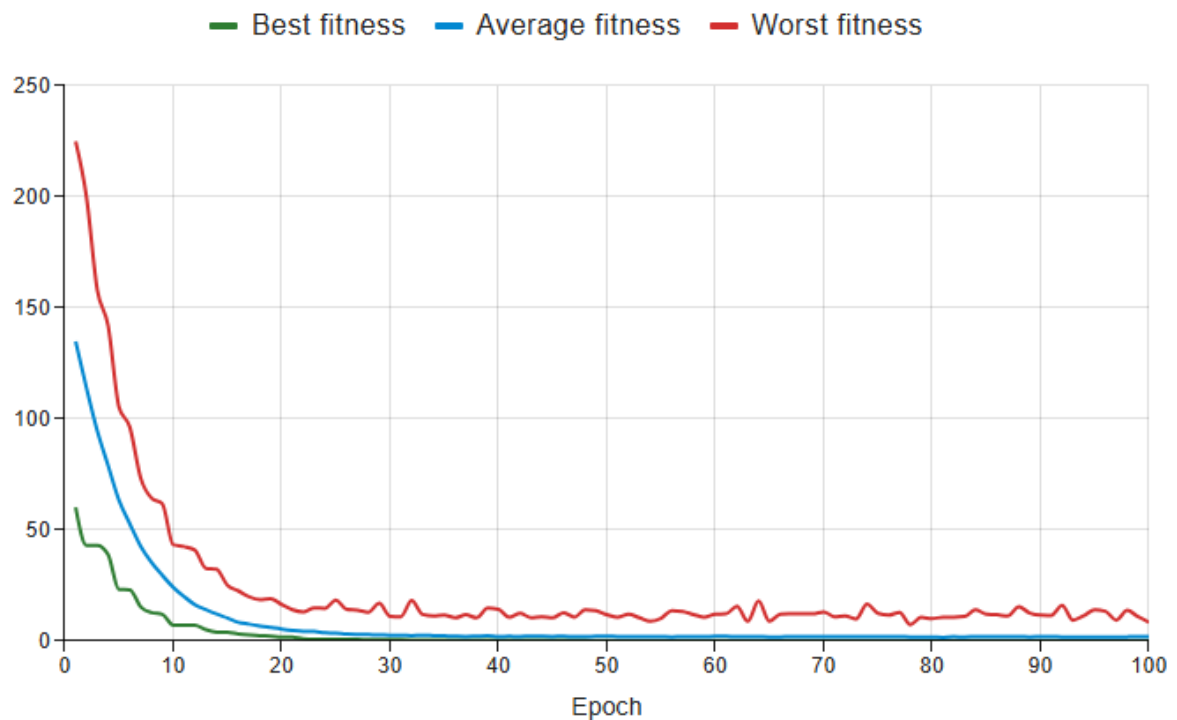
Ustawienia :

Reprezentacja	Binarna	Realna
Liczba zmiennych	20	20
Liczba bitów	20	-
Rozmiar populacji	500	500
Liczba epok	100	100
Metoda selekcji	Tournament	Tournament
Rozmiar turnieju (K)	3	3
Metoda krzyżowania	Two Point	Two Point
Metoda mutacji	Random	Gaussian (P3 spec)
Prawdopodobieństwo krzyżowania	0.8	0.8
Prawdopodobieństwo mutacji	0.01	0.01
Liczba rodziców	250	250
Wątki równoległe	0	0
Strategia elitarna	Włączona	Włączona

Wykresy reprezentacja binarna:



Wykresy reprezentacja realna:



Wnioski:

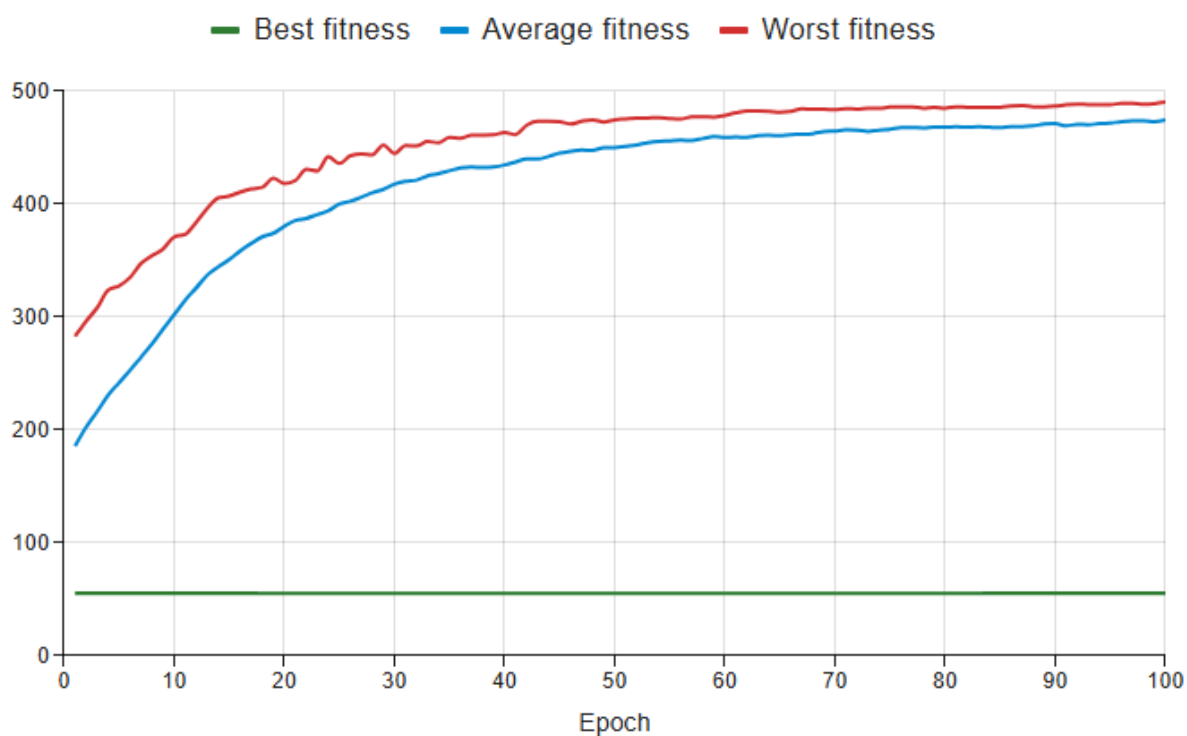
- Wykres binarny pokazuje wyraźną, monotonicznie opadającą krzywą najlepszego przystosowania, jednak duży rozstaw między krzywą najlepszą a średnią sugeruje, że reszta populacji nie nadąża za liderem.
- W wariacie rzeczywistym wszystkie trzy krzywe (best, avg, worst) zbiegają się znacznie bliżej siebie, tworząc bardziej równomierny i stabilny przebieg zbieżności.
- Odchylenie standardowe wartości funkcji przystosowania (fitness), wykreślane per generacja, opada wyraźnie w obu reprezentacjach, potwierdzając, że populacja faktycznie się „ścieśnia” wokół dobrego obszaru.

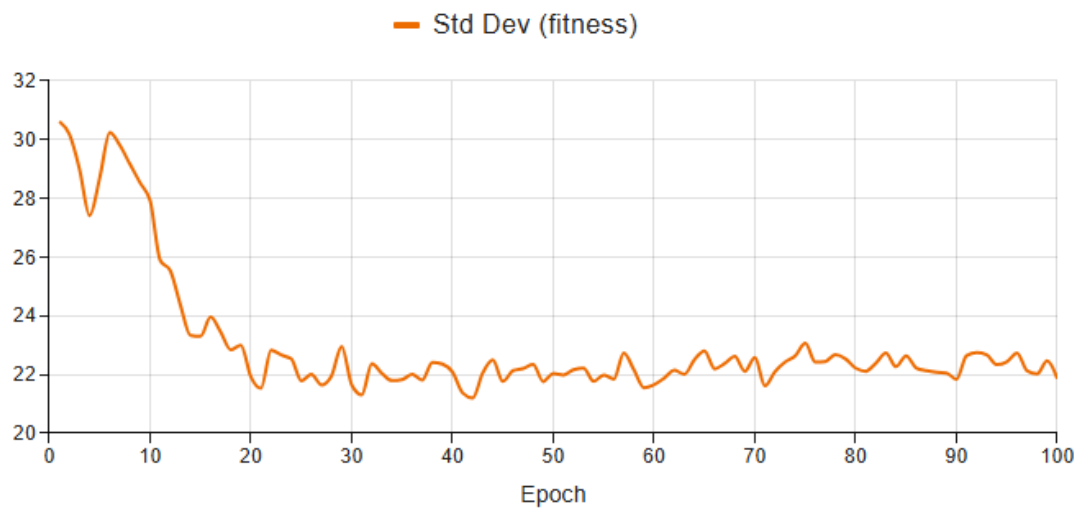
Konfiguracja 2 - Agresywna Eksploatacja

Ustawienia :

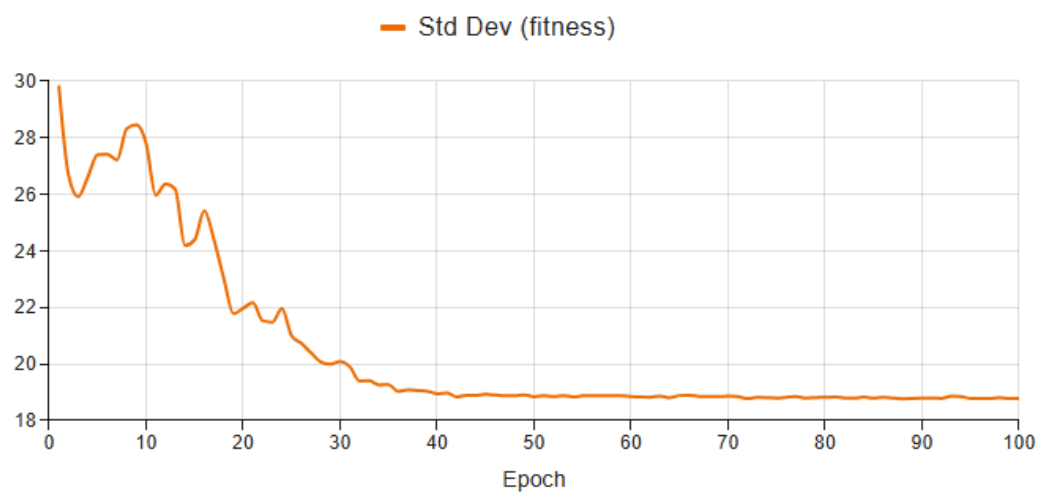
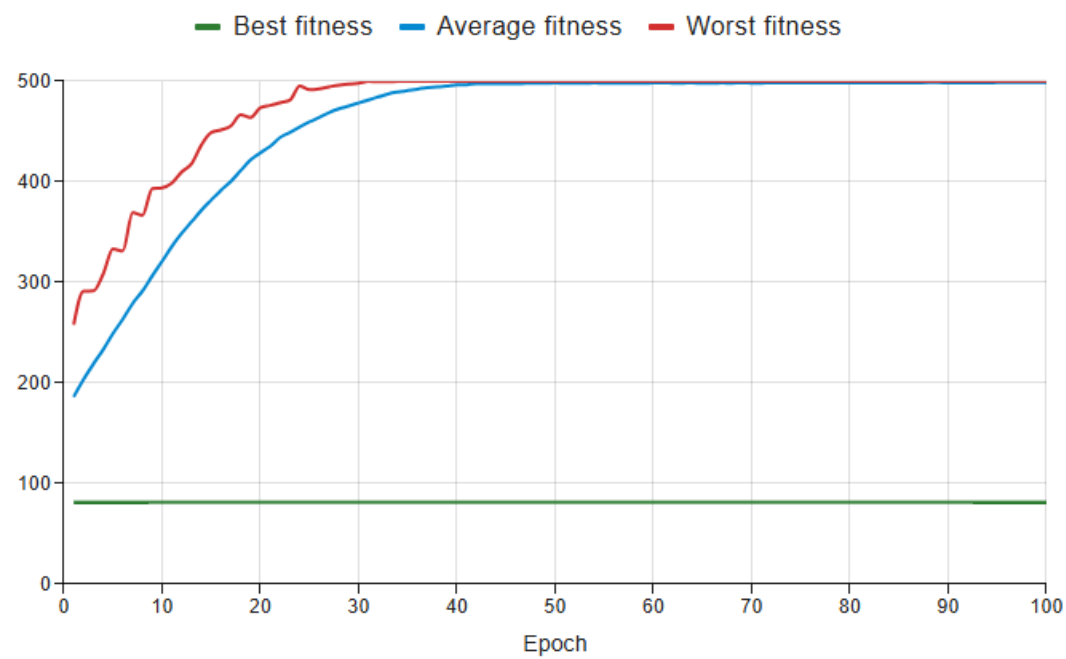
Reprezentacja	Binarna	Realna
Liczba zmiennych	20	20
Liczba bitów	20	-
Rozmiar populacji	500	500
Liczba epok	100	100
Metoda selekcji	Rank	Rank
Rozmiar turnieju (K)	-	-
Metoda krzyżowania	Single Point	Single Point
Metoda mutacji	Random	Gaussian (P3 spec)
Prawdopodobieństwo krzyżowania	0.85	0.85
Prawdopodobieństwo mutacji	0.01	0.01
Liczba rodziców	250	250
Wątki równoległe	0	0
Strategia elitarna	Włączona	Włączona

Wykresy reprezentacja binarna:





Wykresy reprezentacja realna:



Wnioski:

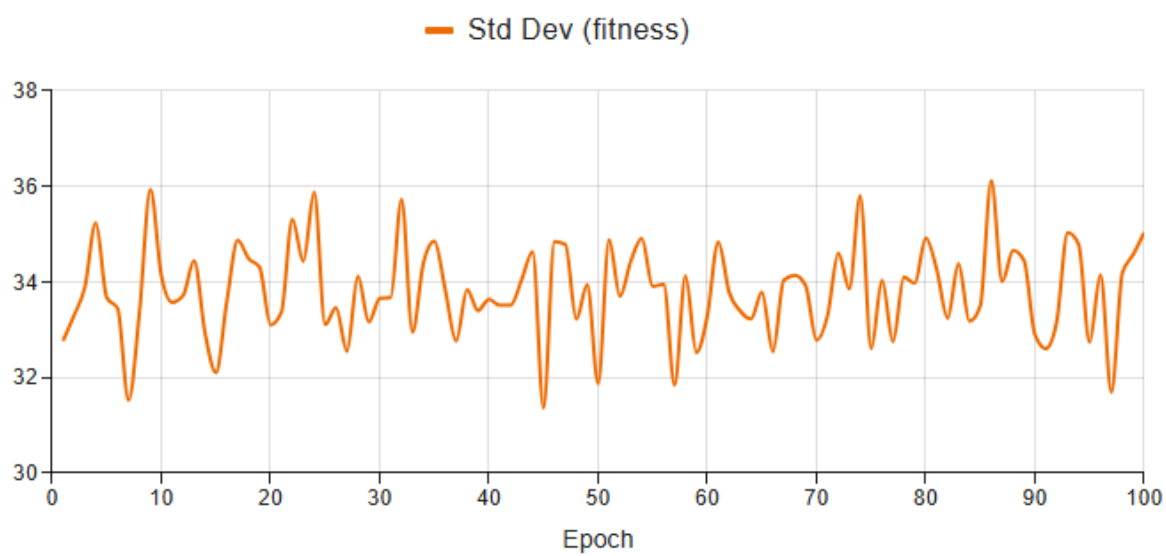
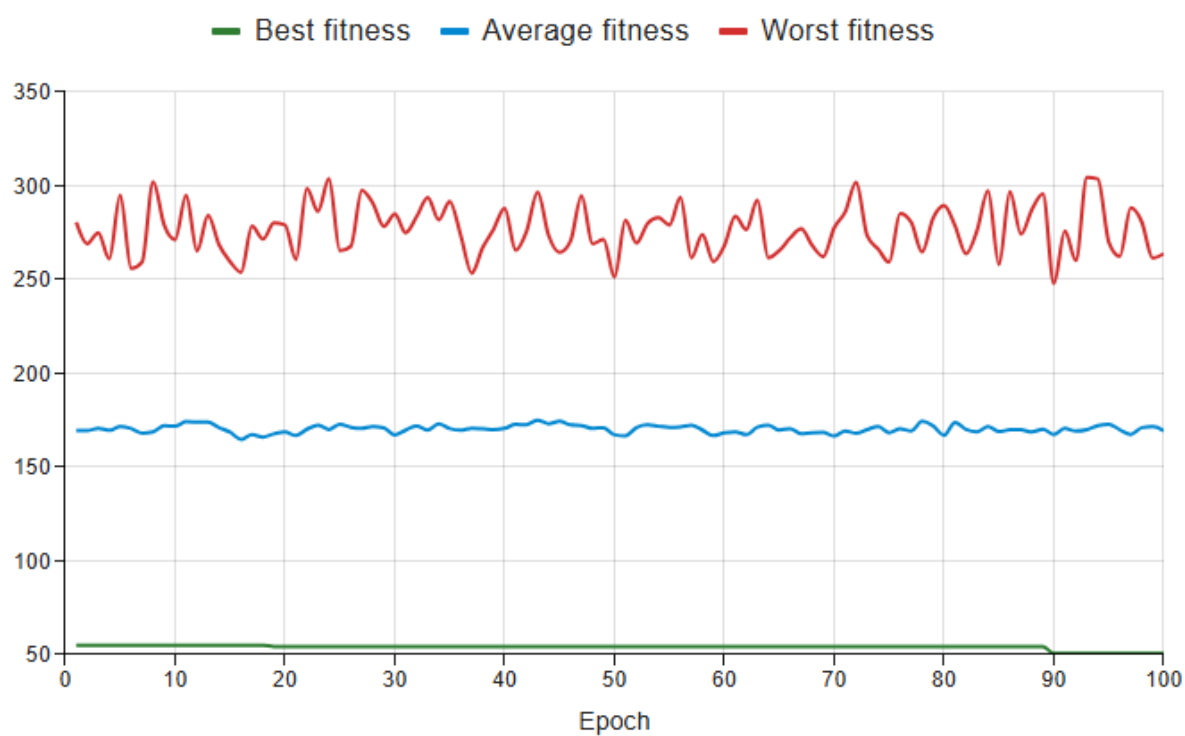
- Wykres binarny pokazuje całkowicie płaską linię najlepszego dopasowania na stałym, wysokim poziomie, co oznacza natychmiastowe utknięcie w lokalnym minimum bez żadnej poprawy.
- Krzywe średnia i najgorsza dla wariantu binarnego gwałtownie pną się w górę, co obrazuje drastyczne i niekontrolowane pogarszanie się jakości pozostałej części populacji.
- W reprezentacji realnej wykres najlepszego dopasowania również pozostaje idealnie płaską linią poziomą, wskazując na identyczny problem przedwczesnej zbieżności na samym początku.
- Krzywe średnia i najgorsza w wariancie realnym bardzo szybko rosną i stabilizują się na maksymalnym poziomie, tworząc ciasny, spłaszczony profil na górze wykresu.
- Odchylenie standardowe w obu reprezentacjach gwałtownie spada w pierwszych epokach, po czym przechodzi w linię poziomą, co pokazuje całkowitą utratę różnorodności i zamrożenie populacji.

Konfiguracja 3 – Maksymalna Eksploracja

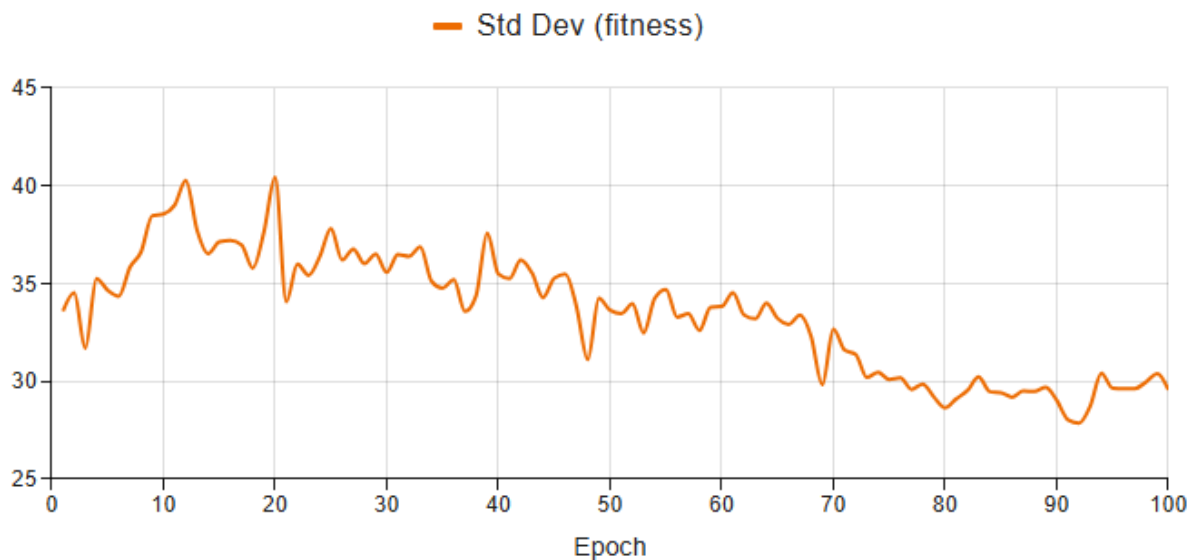
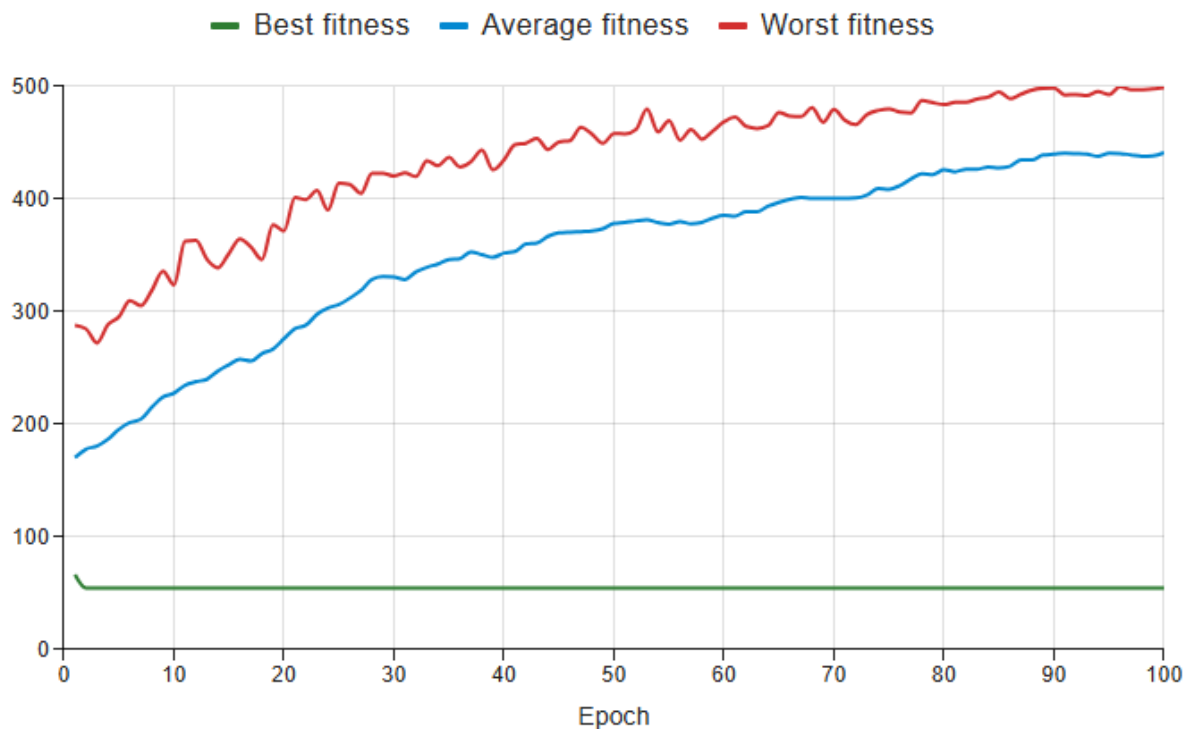
Ustawienia :

Reprezentacja	Binarna	Realna
Liczba zmiennych	20	20
Liczba bitów	20	-
Rozmiar populacji	500	500
Liczba epok	100	100
Metoda selekcji	Roulette	Roulette
Rozmiar turnieju (K)	-	-
Metoda krzyżowania	Uniform	Uniform
Metoda mutacji	Random	Gaussian (P3 spec)
Prawdopodobieństwo krzyżowania	0.9	0.9
Prawdopodobieństwo mutacji	0.15	0.15
Liczba rodziców	250	250
Wątki równoległe	0	0
Strategia elitarna	Włączona	Włączona

Wykresy reprezentacja binarna:



Wykresy reprezentacja realna:



Wnioski:

- Wykres binarny prezentuje idealnie płaską linię najlepszego osobnika, natomiast krzywe średnia i najgorsza silnie i chaotycznie falują na wysokim poziomie przez całe 100 epok.
- W wariancie rzeczywistym linia najlepszego dopasowania zalicza lekkie tąpnięcie na samym początku, po czym stabilizuje się jako linia prosta.
- Krzywe średnia i najgorsza dla reprezentacji realnej stale i płynnie pną się w górę, co pokazuje, że wysoka mutacja uniemożliwia populacji zbliżenie się do lidera.
- Wykres odchylenia standardowego dla wersji binarnej przypomina bardzo poszarpany, nieustanny zygzak, co potwierdza ciągłe wprowadzanie chaosu przez agresywne operatory.

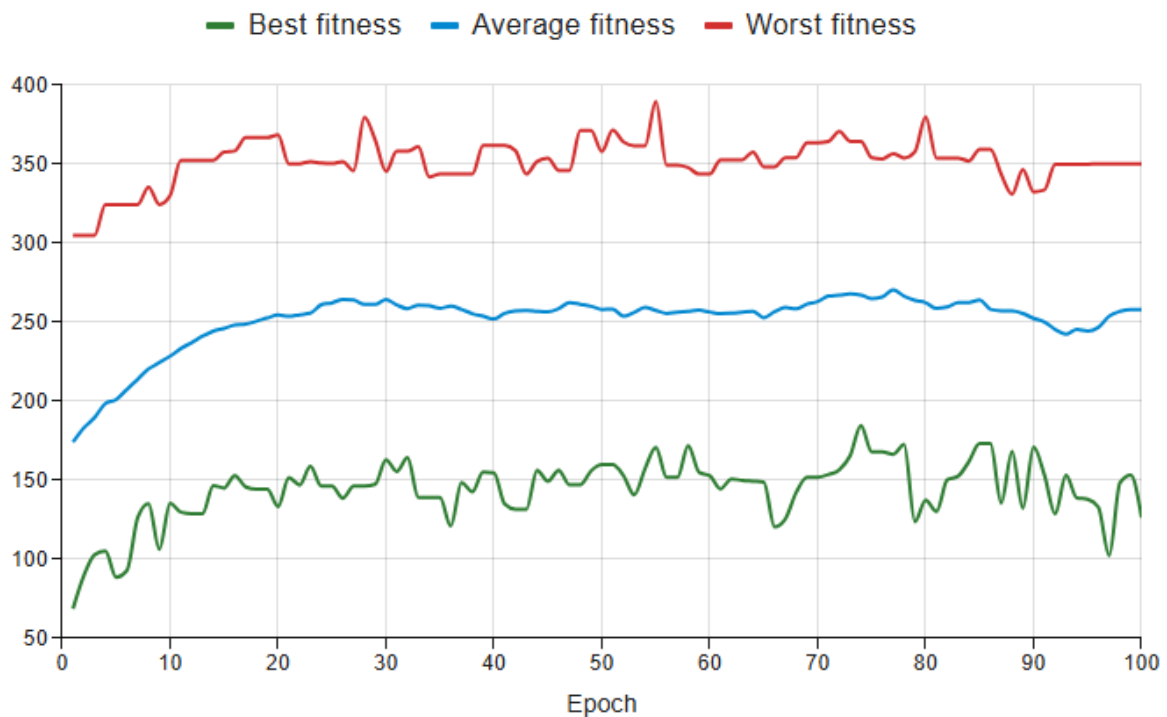
- W wariancie realnym odchylenie standardowe po początkowych wahaniach zaczyna powoli i systematycznie opadać, co sugeruje powolne, ale bardzo odporne porządkowanie populacji.

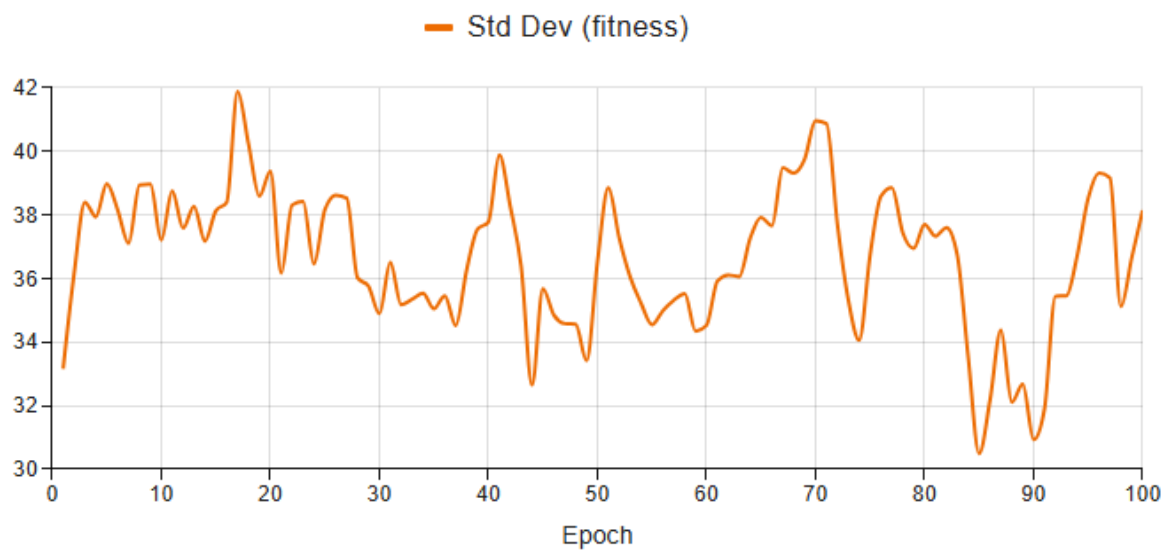
Konfiguracja 4 – Test bez zabezpieczeń

Ustawienia :

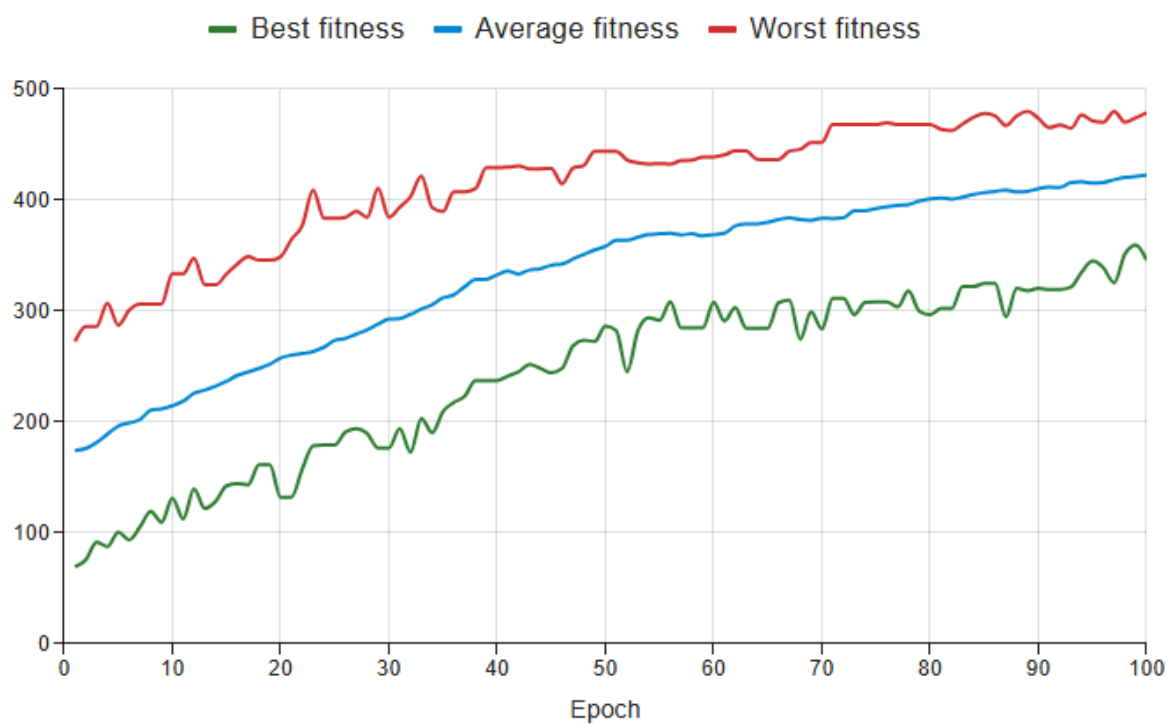
Reprezentacja	Binarna	Realna
Liczba zmiennych	20	20
Liczba bitów	20	-
Rozmiar populacji	500	500
Liczba epok	100	100
Metoda selekcji	Roulette	Roulette
Rozmiar turnieju (K)	-	-
Metoda krzyżowania	One point	One point
Metoda mutacji	Random	Gaussian (P3 spec)
Prawdopodobieństwo krzyżowania	0.7	0.7
Prawdopodobieństwo mutacji	0.05	0.05
Liczba rodziców	250	250
Wątki równoległe	0	0
Strategia elitarna	Wyłączona	Wyłączona

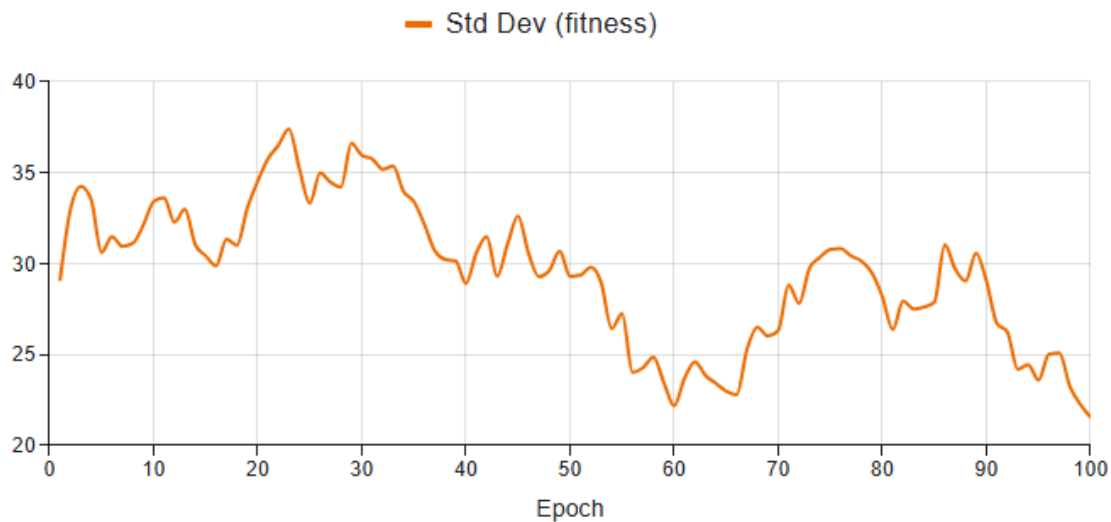
Wykresy reprezentacja binarna:





Wykresy reprezentacja realna:





Wnioski:

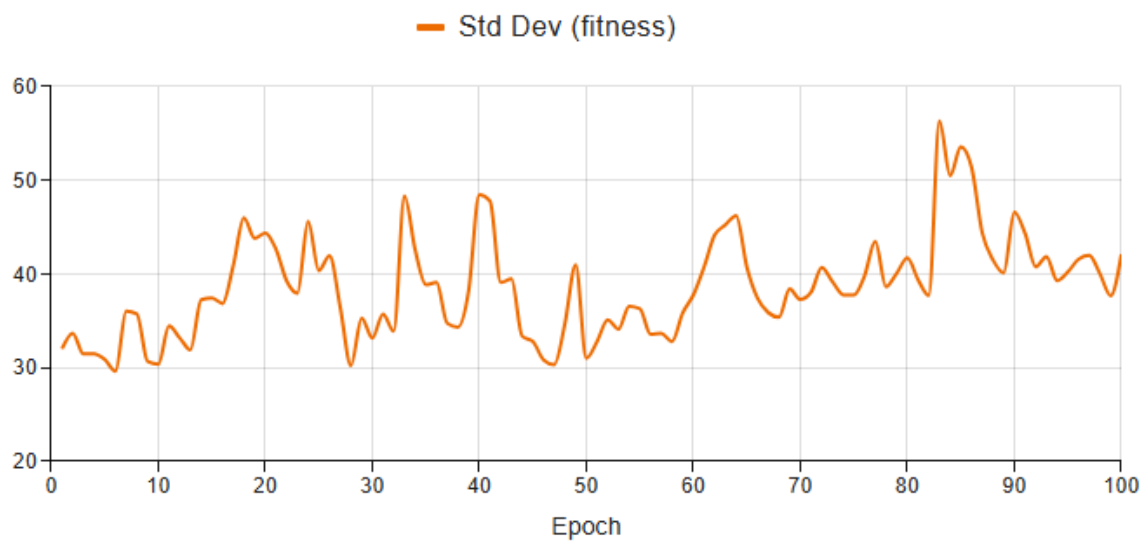
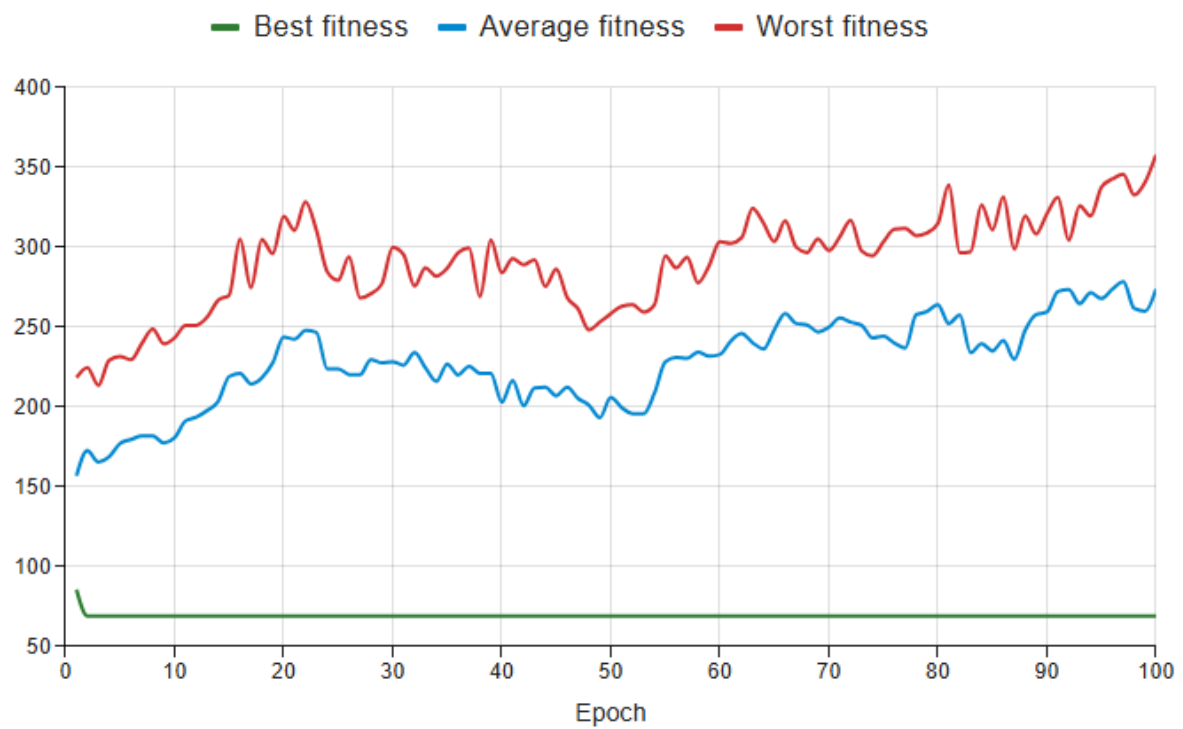
- Wykres binarny dla najlepszego osobnika charakteryzuje się bardzo niestabilnym, silnie poszarpanym przebiegiem z licznymi skokami w górę i w dół, obrazując brak ochrony najlepszych genów.
- Krzywe średnia i najgorsza w reprezentacji binarnej poruszają się falami w szerokim rozstępie, idealnie odwzorowując niestabilność lidera.
- W wariancie realnym krzywa najlepszego dopasowania ma schodkowy, falujący przebieg, który zamiast opadać do zera, sukcesywnie pnie się w stronę gorszych wartości.
- Krzywe średnia i najgorsza dla reprezentacji realnej wznoszą się bardzo szerokim łukiem, co pokazuje całkowity rozpad populacji i brak jakiegokolwiek zbieżności.
- Wykres odchylenia standardowego w obu wariantach jest skrajnie chaotyczny, pełen głębokich dolin i wysokich szczytów, co potwierdza permanentną utratę stabilności algorytmu.

Konfiguracja 5 – Mała populacja

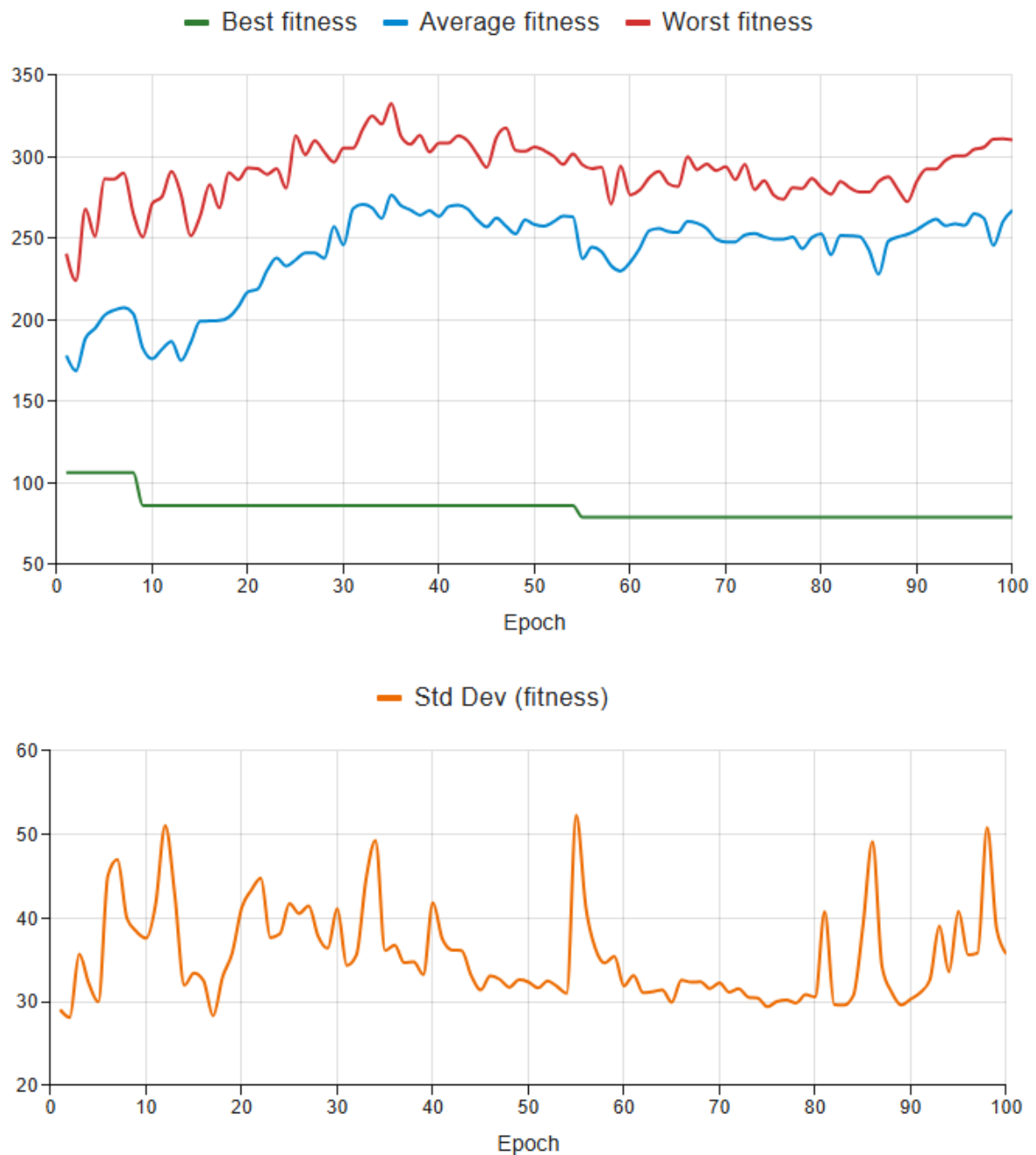
Ustawienia :

Reprezentacja	Binarna	Realna
Liczba zmiennych	20	20
Liczba bitów	20	-
Rozmiar populacji	40	40
Liczba epok	100	100
Metoda selekcji	Roulette	Roulette
Rozmiar turnieju (K)	-	-
Metoda krzyżowania	Two point	Two point
Metoda mutacji	Random	Gaussian (P3 spec)
Prawdopodobieństwo krzyżowania	0.8	0.8
Prawdopodobieństwo mutacji	0.02	0.02
Liczba rodziców	20	20
Wątki równoległe	0	0
Strategia elitarna	Wyłączona	Wyłączona

Wykresy reprezentacja binarna:



Wykresy reprezentacja realna:



Wnioski:

- Wykres najlepszego dopasowania dla reprezentacji binarnej to idealnie płaska linia z lekkimi schodkami w dół, pod którą krzywe średnia i najgorsza poruszają się chaotycznymi, powtarzalnymi falami.
- W wariancie realnym linia najlepszego osobnika ma postać rzadkich, ostrych stopni w dół, co pokazuje, że poprawa następuje tylko raz na kilkadziesiąt epok.
- Krzywe średnia i najgorsza dla chromosomu realnego bardzo mocno i gwałtownie falują, tworząc dynamiczny, niestabilny profil przez cały czas trwania algorytmu.

- Wykres odchylenia standardowego dla obu reprezentacji ma formę bardzo wysokich, nieregularnych i ostrych szczytów, co jest bezpośrednim skutkiem losowości w miniaturowej populacji.

Porównanie wyników dla różnych konfiguracji algorytmu. Własna implementacja vs PyGAD

Chromosom binarny

Konfiguracja	Najlepszy wynik (Best Fitness)	Najlepszy wynik (Best Fitness) PyGAD	Średni wynik populacji	Średni wynik populacji PyGAD	Czas wykonania (binarny) (s)	Czas wykonania (s) PyGAD
Konfiguracja Zbalansowana	0.0015	0.0062	0.0062	3.4685	4.55	30.06
Agresywna Eksploatacja (Best Selection)	19.29	55.29	474.44	293.97	3.54	29.98
Maksymalna Eksploracja (Wysoka mutacja)	8.26	50.67	50.82	169.51	7.12	83.25
Test bez zabezpieczeń (Brak Elitaryzmu)	50.11	126.46	193.53	258.08	4.23	27.89
Mała populacja (40 osobników)	7.46	69.06	11.76	293.97	0.34	2.56

Chromosom realny

Konfiguracja	Najlepszy wynik (Best Fitness)	Najlepszy wynik (Best Fitness) PyGAD	Średni wynik populacji	Średni wynik populacji PyGAD	Czas wykonania (binarny) (s)	Czas wykonania (s) PyGAD
Konfiguracja Zbalansowana	0.0026	0.1781	0.22039	1.8941	1.4004	10.6449
Agresywna Eksploatacja (Best Selection)	0.5058	81.5067	1.0323	498.6548	0.7457	10.8371
Maksymalna Eksploracja (Wysoka mutacja)	23.6365	54.7654	127.0588	441.3699	1.0744	10.3198
Test bez zabezpieczeń (Brak Elitaryzmu)	36.5429	345.7156	92.1593	422.5881	0.9221	8.1660
Mała populacja (40 osobników)	0.6536	79.1663	1.663	267.4439	0.0649	0.8068

Analiza porównawcza i wnioski:

- **Całkowita dominacja własnej implementacji** – autorski algorytm genetyczny okazał się pod każdym względem bezkonkurencyjny, deklasując bibliotekę PyGAD zarówno pod kątem drastycznie krótszego czasu wykonywania obliczeń, jak i nieporównywalnie wyższej precyzji w odnajdywaniu globalnego optimum funkcji sferycznej.
- **Narzut proceduralny i generyczność PyGAD** – tak długie czasy pracy biblioteki wynikają bezpośrednio z jej uniwersalnej architektury, która w każdej epoce marnuje potężne zasoby procesora na setki wewnętrznych weryfikacji, sprawdzanie typów danych, granic przedziałów oraz obsługę zdarzeń, podczas gdy własny kod jest prosty, liniowy i uszyty bezpośrednio pod jedno konkretne zadanie.
- **Paraliż wydajnościowy przy wysokiej mutacji** – Konfiguracja 3 z agresywną mutacją obnażyła krytyczną wadę PyGAD, gdzie konieczność ciągłej alokacji nowej pamięci w strukturach macierzowych doprowadziła do paraliżu czasu pracy (aż 83.25 sekund w wariancie binarnym), podczas gdy własna, prosta implementacja przeszła przez ten proces bez odczuwalnego wysiłku obliczeniowego.
- **Podatność biblioteki na przedwczesną zbieżność** – wyniki liczbowe jasno pokazują, że gotowe mechanizmy replikacji w PyGAD bardzo szybko doprowadzały do całkowitego zamrożenia populacji, przez co biblioteka trwale utykała w fatalnych minimach lokalnych (np. Best Fitness na poziomie 81.50 w Konfiguracji 2), podczas gdy własny algorytm elastycznie parł do przodu.
- **Całkowity rozpad struktur bez elitaryzmu** – usunięcie strategii elitarniej (Konfiguracja 4) wywołało w strukturach PyGAD absolutny chaos obliczeniowy i brak jakiegokolwiek zbieżności, co skutkowało drastycznym pogorszeniem wyniku końcowego (Best Fitness 345.71) w porównaniu do stabilnej i odpornej na brak zabezpieczeń autorskiej wersji.

- **Zupełna bezużyteczność PyGAD w małej skali** – test miniaturowej populacji (Konfiguracja 5) udowodnił, że przy małej liczbie osobników narzut biblioteki całkowicie zabija jej ewolucyjny sens, generując potężne opóźnienia i losowe wyniki (Best Fitness 79.16), podczas gdy własny, lekki kod zrealizował to zadanie w ułamku sekundy z bliskim ideału rezultatem.
- **Wpływ filozofii minimalizacji na dynamikę zbieżności** – natywne przystosowanie własnego kodu do problemu minimalizacji pozwoliło populacji na prowadzenie poszukiwań najkrótszą i najbardziej naturalną ścieżką, podczas gdy PyGAD – jako narzędzie domyślnie maksymalizujące – musiał stosować sztuczną negację funkcji celu, co wprowadziło dodatkową warstwę obliczeniową i negatywnie wpłynęło na trajektorię zbieżności.